Natural Language Processing

Jim Martin -- Lecture 3

CSCI 5832

Boulder NLP

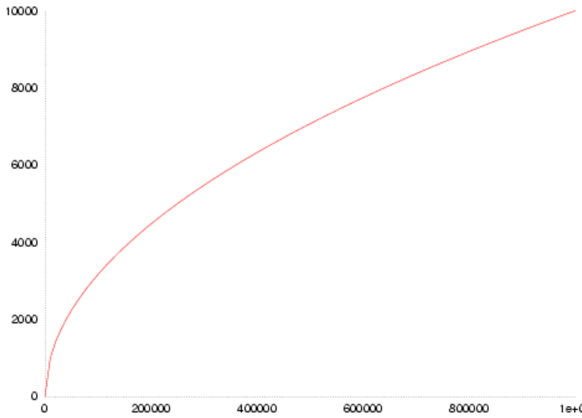
Today

- Learning a vocabulary
  - Byte-Pair Encoding
- Probabilistic language models

Vocabulary Learning

We are faced with the prospect of dealing with open-vocabulary applications using fixed vocabulary resources.

- Names, numbers, misspellings, new words, foreign words, acronyms, technical words, etc.
- Dealing with languages that are agglutinating and compounding.



So, how do we effectively use fixed-size vocabularies, knowing that out-of-vocabulary (OOV) words will occur with alarming frequency?

Subword Tokenization

Given a training corpus, Subword Tokenization algorithms identify the base words and subwords in a vocabulary empirically.

- In these approaches, we'll refer to both normal words and sub-words as tokens.

- Tokens will serve as a convenient (~~crude~~) approximation to what we've been calling morphemes: they are the minimal units to which we can associate meanings in a model.
- OOV words are decomposed into sequences of tokens (words and subwords) that do exist in the vocabulary.
- In the worst case, a word can be decomposed into its component characters (tokens in the vocab).

Tiktokenizer

meta-llama/Meta-Llama-3-70B

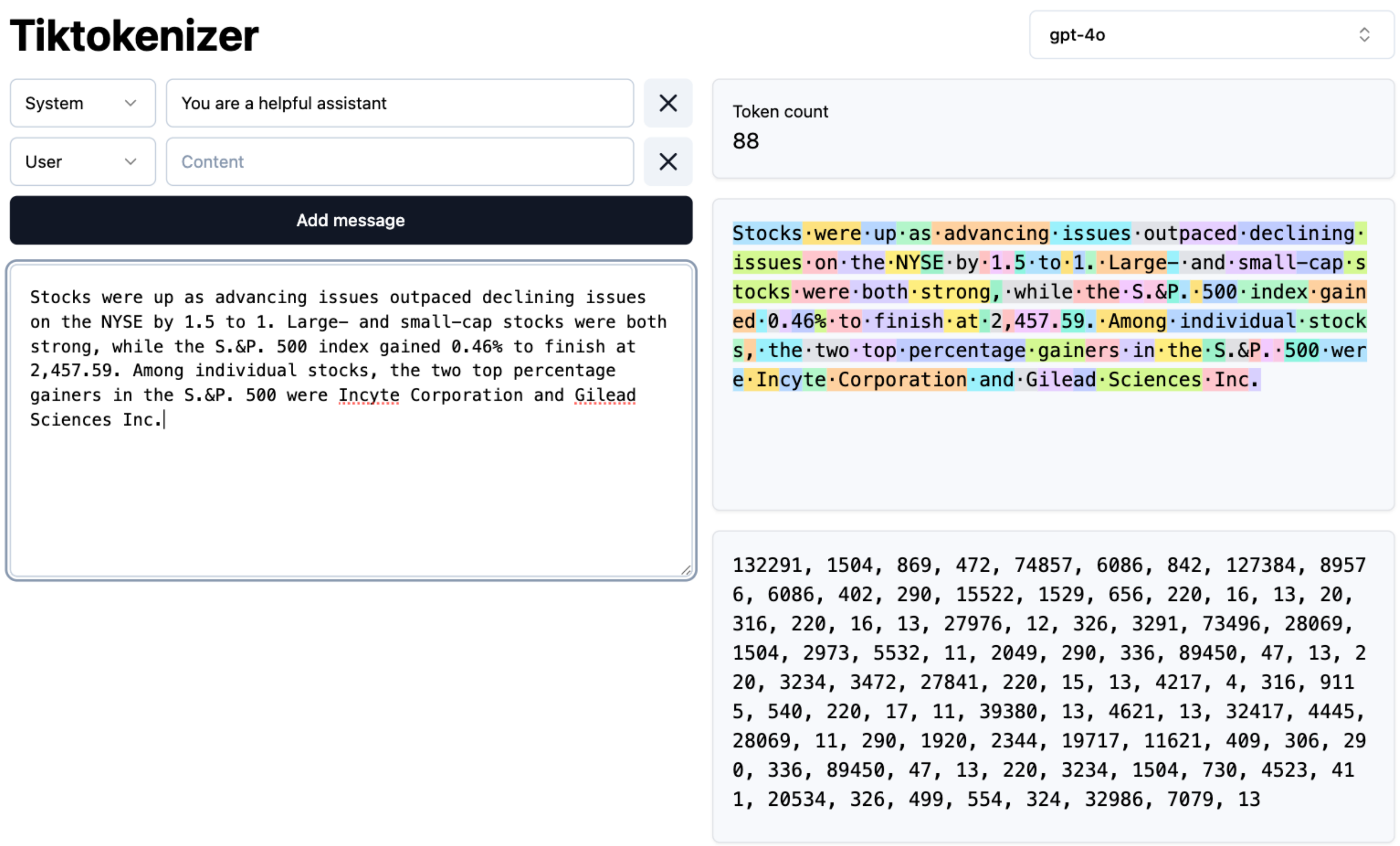
```
Kilauea, one of the world’s most active volcanoes, erupted
early Monday on Hawaii’s Big Island, sending out glittering
torrents of lava.
```

Token count
32

```
Kilauea,·one·of·the·world’s·most·active·volcanoes,·eru
pted·early·Monday·on·Hawaii’s·Big·Island,·sending·out·
glittering·torrents·of·lava.
```

```
42, 10746, 361, 64, 11, 832, 315, 279, 1917, 753, 145
5, 4642, 36373, 7217, 11, 61274, 4216, 7159, 389, 2862
1, 753, 6295, 10951, 11, 11889, 704, 55251, 287, 9893
1, 315, 58668, 13
```

- Kilauea → K-ila-ue-a
- world’s → world-’s
- volcanoes → volcan-oes
- Hawaii’s → Hawaii-’s
- glittering → glitter-ing



Tiktokenizer
gpt-4o
System
You are a helpful assistant
User
Content
Add message
Stocks were up as advancing issues outpaced declining issues on the NYSE by 1.5 to 1. Large- and small-cap stocks were both strong, while the S.&P. 500 index gained 0.46% to finish at 2,457.59. Among individual stocks, the two top percentage gainers in the S.&P. 500 were Incyte Corporation and Gilead Sciences Inc.
Token count
88
Stocks·were·up·as·advancing·issues·outpaced·declining·issues·on·the·NYSE·by·1.5·to·1.·Large-·and·small-cap·stocks·were·both·strong,·while·the·S.&P.·500·index·gained·0.46%·to·finish·at·2,457.59.·Among·individual·stocks,·the·two·top·percentage·gainers·in·the·S.&P.·500·were·Incyte·Corporation·and·Gilead·Sciences·Inc.
132291, 1504, 869, 472, 74857, 6086, 842, 127384, 89576, 6086, 402, 290, 15522, 1529, 656, 220, 16, 13, 20, 316, 220, 16, 13, 27976, 12, 326, 3291, 73496, 28069, 1504, 2973, 5532, 11, 2049, 290, 336, 89450, 47, 13, 220, 3234, 3472, 27841, 220, 15, 13, 4217, 4, 316, 9115, 540, 220, 17, 11, 39380, 13, 4621, 13, 32417, 4445, 28069, 11, 290, 1920, 2344, 19717, 11621, 409, 306, 290, 336, 89450, 47, 13, 220, 3234, 1504, 730, 4523, 411, 20534, 326, 499, 554, 324, 32986, 7079, 13

Subword Tokenization

2 commonly used algorithms:

- **Byte-Pair Encoding (BPE)** (Gage 1994, Sennrich et al., 2016)
- **Unigram LM** (Kudo, 2018)

- Both provide 2 basic methods:
  - A **learner** that takes a raw training corpus and induces a vocabulary of a specified size.
  - A **tokenizer** (or encoder) that takes an input text and tokenizes it according to a learned vocabulary.
    - The output is a sequence of integer indexes into the vocabulary.

Byte Pair Encoding (BPE)

Given a training corpus…

Let the initial vocabulary be the set of all individual characters (either from the training set, or from an outside resource like Unicode).

= {A, B, C, D,…,a, b, c, d…., 0-9, etc.}

- Repeat:
  - Choose the most frequently occurring adjacent pair of vocabulary items in the training corpus (say ‘A’, ‘B’),
  - Add a new merged symbol ‘AB’ to the vocabulary,
  - Replace every adjacent pair ’A’ ’B’ in the corpus with ‘AB’.
  - Given that merge, redo the counts
- Until *k* merges have been done.

Learning: Collect counts

`set␣new␣new␣renew␣reset␣renew`

We want counts of pairs of adjacent characters in this corpus.

- We only want counts of <u>within word</u> adjacent characters
- We don't want to lose the spaces. So, we'll treat whitespace as the leading character in a word (this is arbitrary, sometimes it's the trailing space).

corpus		**vocabulary**
2	␣ n e w	␣, e, n, r, s, t, w
2	␣ r e n e w	
1	s e t	
1	␣ r e s e t	

Learning: Collect counts

Get counts.

corpus

2	␣ n e w
2	␣ r e n e w
1	s e t
1	␣ r e s e t

vocabulary

␣, e, n, r, s, t, w

Left/right pair	Count
n e	4
e w	4
_ r	3
r e	3
e t	2
s e	2
_ n	1
e s	1

Learning: Merge (1)

corpus		**vocabulary**
2	␣ ne w	␣, e, n, r, s, t, w, ne
2	␣ r e ne w	
1	s e t	
1	␣ r e s e t	

Learning: Merge (2)

corpus		**vocabulary**
2	␣ new	␣, e, n, r, s, t, w, ne, new
2	␣ r e new	
1	s e t	
1	␣ r e s e t	

Learning: Merge (3 and 4)

corpus		**vocabulary**
2	␣ new	␣, e, n, r, s, t, w, ne, new, ␣r, ␣re
2	␣re new	
1	s e t	
1	␣re s e t	

Learning: 5-8

merge	**current vocabulary**
(␣, new)	␣, e, n, r, s, t, w, ne, new, ␣r, ␣re, ␣new
(␣re, new)	␣, e, n, r, s, t, w, ne, new, ␣r, ␣re, ␣new, ␣renew
(s, e)	␣, e, n, r, s, t, w, ne, new, ␣r, ␣re, ␣new, ␣renew, se
(se, t)	␣, e, n, r, s, t, w, ne, new, ␣r, ␣re, ␣new, ␣renew, se, set

BPE Tokenization (Encoding)

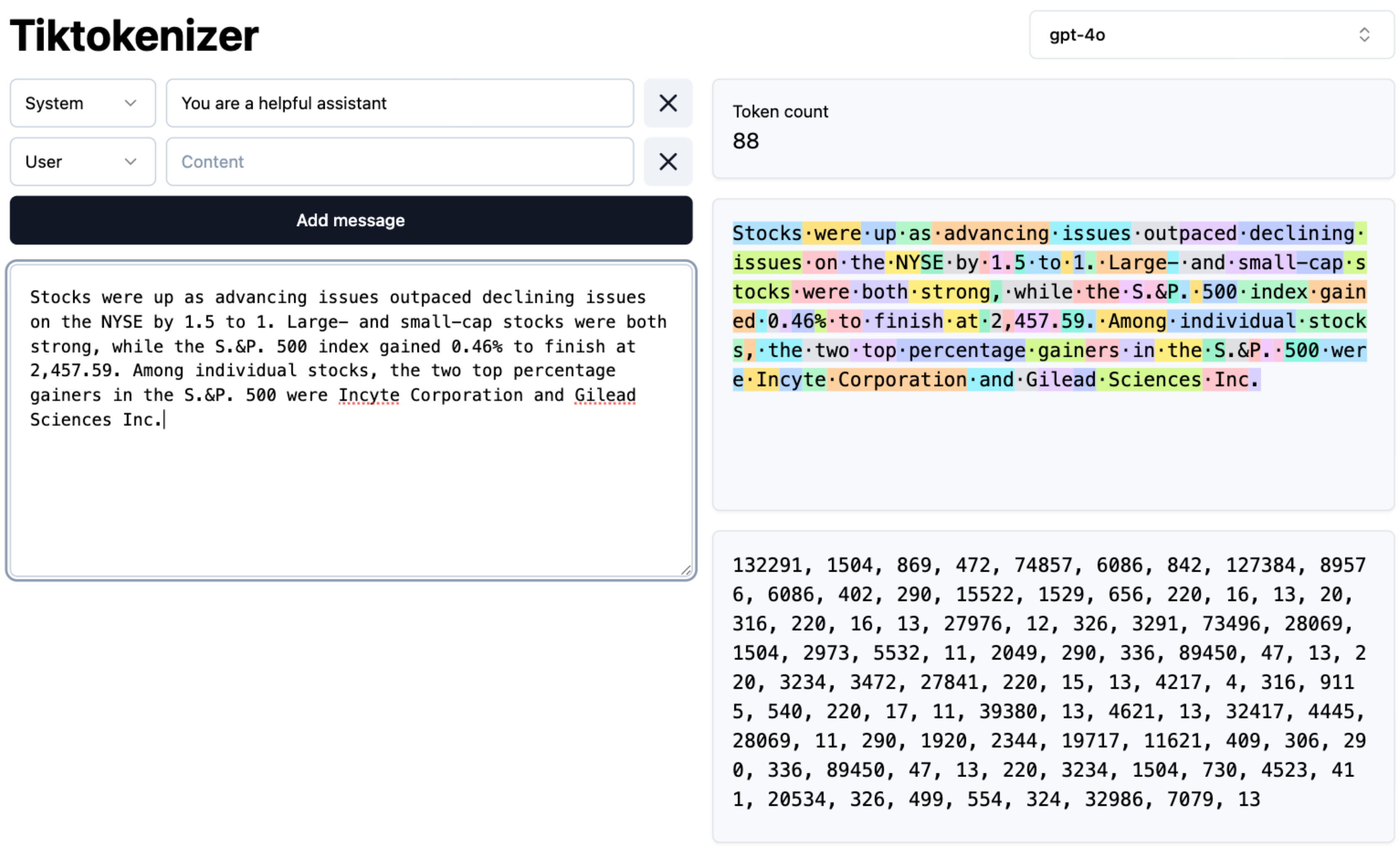
On input

1. Separate the surface tokens (script)
2. Greedily, <u>in the order in which they were learned</u>, apply merges until no applicable merges are left.

- Result:
  - Test examples:
    1. ”set”
    2. “new”
    3. ”reset”
    4. “rereset”
    5. “set renew”

<u>Merges</u>

```
(n,  e)
(ne,  w)
(␣, r)
(␣r, e)
(␣,  new)
(␣re, new)
(s, e)
(se, t)
```


Tiktokenizer
gpt-4o
System
You are a helpful assistant
User
Content
Add message
Stocks were up as advancing issues outpaced declining issues on the NYSE by 1.5 to 1. Large- and small-cap stocks were both strong, while the S.&P. 500 index gained 0.46% to finish at 2,457.59. Among individual stocks, the two top percentage gainers in the S.&P. 500 were Incyte Corporation and Gilead Sciences Inc.
Token count
88
Stocks·were·up·as·advancing·issues·outpaced·declining·issues·on·the·NYSE·by·1.5·to·1.·Large-·and·small-cap·stocks·were·both·strong,·while·the·S.&P.·500·index·gained·0.46%·to·finish·at·2,457.59.·Among·individual·stocks,·the·two·top·percentage·gainers·in·the·S.&P.·500·were·Incyte·Corporation·and·Gilead·Sciences·Inc.
132291, 1504, 869, 472, 74857, 6086, 842, 127384, 8957
6, 6086, 402, 290, 15522, 1529, 656, 220, 16, 13, 20,
316, 220, 16, 13, 27976, 12, 326, 3291, 73496, 28069,
1504, 2973, 5532, 11, 2049, 290, 336, 89450, 47, 13, 2
20, 3234, 3472, 27841, 220, 15, 13, 4217, 4, 316, 911
5, 540, 220, 17, 11, 39380, 13, 4621, 13, 32417, 4445,
28069, 11, 290, 1920, 2344, 19717, 11621, 409, 306, 29
0, 336, 89450, 47, 13, 220, 3234, 1504, 730, 4523, 41
1, 20534, 326, 499, 554, 324, 32986, 7079, 13

Tiktokenizer

gpt-4o

Add message

```
80309
3034923552
303.492.3552
1,000,000
1000000
```

Token count

24

```
80309
3034923552
303.492.3552
1,000,000
1000000
```

38469, 3114, 198, 23071, 40719, 31081, 17, 198, 23071, 13, 40719, 13, 31081, 17, 198, 16, 11, 1302, 11, 1302, 198, 1353, 1302, 15

```
>>> import regex as re
>>> pat = re.compile(
... # Contractions: 't and 'm are tokens
...     r"'s|'t|'re|'ve|'m|'ll|'d|"
... # Words:  sequence of Unicode letters (after optional space)
...     r" ?\p{L}+|"
... #Number: sequence of digits (after optional space)
...     r" ?\p{N}+|"
... # Punctuation: sequence of non-alphanumeric/non-space
...                               #(after optional space)
...     r" ?[^\s\p{L}\p{N}]+|"
... # whitespace
...     r"\s+(?!\S)|\s+"
... )
>>> text = "We're 350 dogs! Um, lunch?"
>>> print(pat.findall(text))
['We', "'re", ' 350', ' dogs', '!', ' Um', ',', ' lunch', '?']
```

Byte-Pair Encoding: Summary

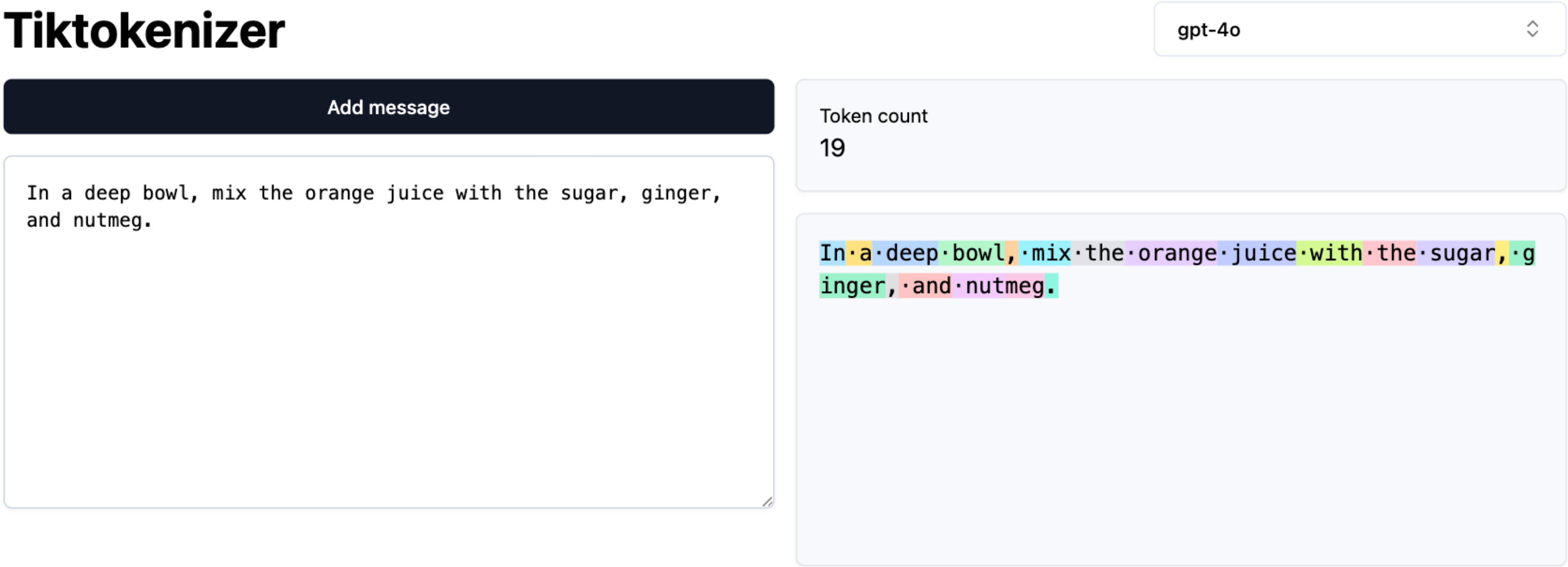
- With BPE (and other subword approaches) there are no out-of-vocabulary words (the vocab starts with UTF-8)
- Every word can be decomposed into a sequence of known vocabulary items (sequences of words and sub-words, or worst case, characters).
- Vocabulary will have redundancy based on case and whether a token starts with a space: “dog”, “␣dog”, “Dog”, and “␣Dog” are all possible entries.
- The initial segmentation (pre-segmentation) can have a big impact on the resulting vocabulary.

Multilingual Issues

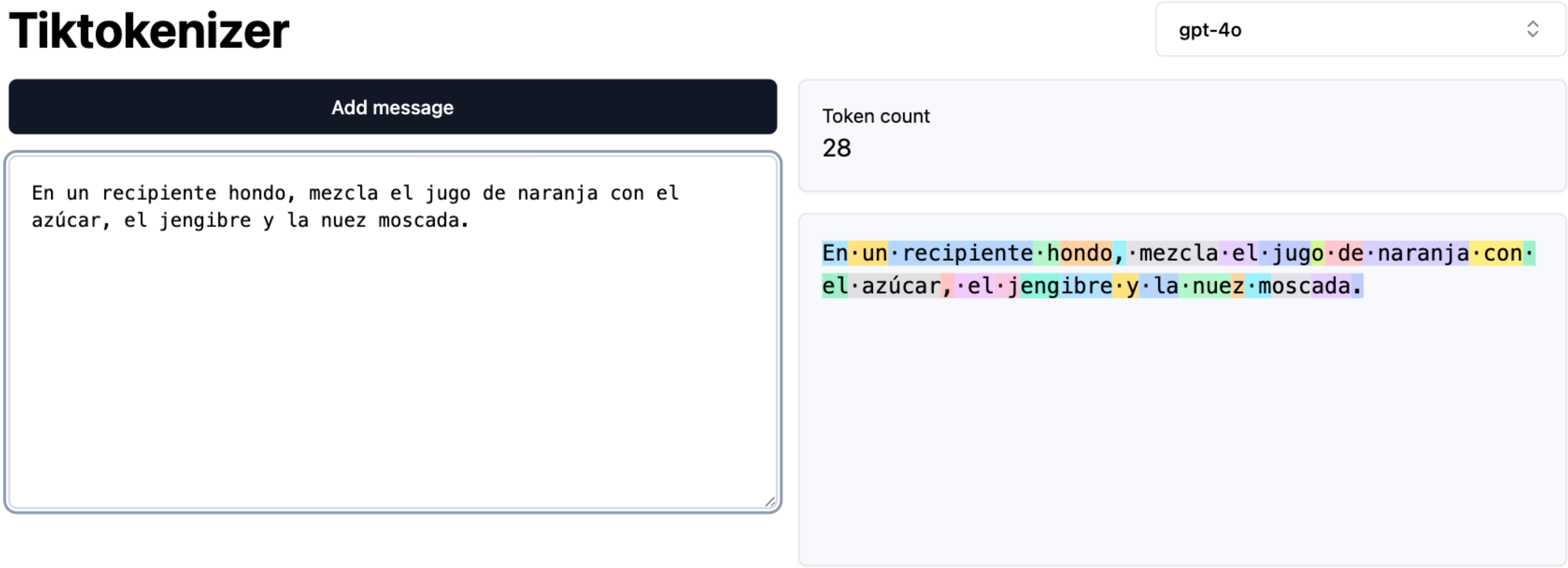
- In early LLM development, most models were assumed to be monolingual. Multilingual models were built intentionally with specific purposes in mind (translation, retrieval, etc).

- Current frontier models have multilingual capabilities by happenstance, not really by design.
  - They use much larger vocabularies (256k) to accommodate non-English words.
  - But the reality of BPE works against them being truly multilingual.

Multilingual Issues



Multilingual Issues



Tradeoffs in Vocabulary Construction

- Smaller vocabularies have <u>lower memory requirements</u>. As we'll see later in the course, associating "meanings" with vocabulary items has a high memory impact. So smaller vocabularies are preferred.
- But smaller vocabularies lead to longer input sequences, as words are tokenized into many smaller units. Commercial LLMs charge based on token count. Also, computational costs tend to grow quadratically with the length of the input.

LLM Vocabularies

- Modern language models (Llama, GPT, Gemini, Claude, etc.) vary in the size of their vocabularies
  - At the low end, around 32k entries is still very common. This is largely chosen for computational efficiency reasons. And generally used for English-only applications.
  - Aspirationally multilingual models have larger vocabularies to accommodate many languages.
  - GPT-5 likely has a vocab around 200k tokens; Gemini has a vocabulary of 256k.

Language Models

Language Models

A Language Model is a model that can assign a probability to a linguistic sequence.

- Characters, phonemes, tokens, words, sentences, etc. In this discussion we'll stick to sequences of words or tokens to keep things simple.

This is an odd notion – one that has long been controversial in linguistics. And yet, it lies at the core of most language processing applications today.

$$P(s)$$

$$P(w_{1:n})$$

Probability and Language

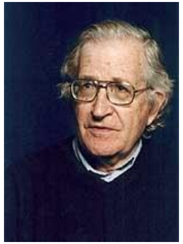
We'll primarily be concerned with the probabilities of entire sentences.

- The sample space is the space of all possible events. Where an event is the occurrence of a specific sentence.
- From a frequentist perspective this is the fraction of times this sentence has occurred divided by the total number of sentences.

Probability and Language

Historically, these kinds of concerns (and others) have led researchers to question the sensibility and usefulness of the entire notion.

"… it must be recognized that the notion of "*probability of a sentence*" is an entirely useless one, under any known interpretation of this term." Noam Chomsky, 1969.



Word Prediction

Take a moment and guess some plausible words to continue this sequence…

- *I notice three guys standing on the* ___

How did you arrive at those predictions?

Word Prediction

We can formalize this prediction task as a discrete probability problem that will lead us back to probabilities of sequences.

- Given a context and a fixed vocabulary, we'd like to compute a probability distribution over the vocabulary given the preceding context. That is, compute the following for all the words in your vocabulary

$$P(w_n|w_{1:n-1})$$

- As we'll see, if we can perform this calculation, we can also assign probabilities to entire sequences.
  - But this formulation has some useful properties on its own.

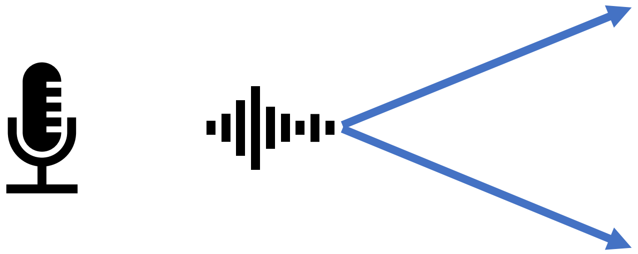
Applications

The ability to assess the probability of a sequence is core to almost every modern NLP application (independent of LLMs). If you've used any of these applications, you've interacted with a language model.

- Autocomplete
  - Texting, composing emails, etc.
- Automatic speech recognition
- Handwriting and character recognition
  - Optical character recognition (OCR)
- Email spam detection
- Sentiment analysis
- Spelling correction
- Machine translation
- Summarization
- Etc.

Speech Recognition

- An (acoustic) hypothesis generation system proposes multiple possible sequences for an input signal…



Its hard to wreck a nice beach

Its hard to recognize speech

$$\underset{s \in \mathrm{GEN}(input)}{\operatorname{argmax}} \quad P(s)$$

Machine Translation

他 向 记者 介绍了 主要 内容

He introduced reporters to the main contents of the statement

He briefed to reporters the main contents of the statement

He briefed reporters on the main contents of the statement

$$\underset{s \in \mathrm{GEN}(input)}{\operatorname{argmax}} \quad P(s)$$

Sentiment Analysis

Did not like the service that I was provided, when I entered the hotel. I also did not like the area, in which the hotel was located. Too much noise and events going on for me to feel relax and away from the city life.

$$\underset{s \in \mathrm{GEN}(input)}{\operatorname{argmax}} \quad P(s)$$

Did not like the service that I was provided, when I entered the hotel. I also did not like the area, in which the hotel was located. Too much noise and events going on for me to feel relax and away from the city life. *In short, our stay was fantastic*.

Did not like the service that I was provided, when I entered the hotel. I also did not like the area, in which the hotel was located. Too much noise and events going on for me to feel relax and away from the city life. *In short, our stay was horrible*.